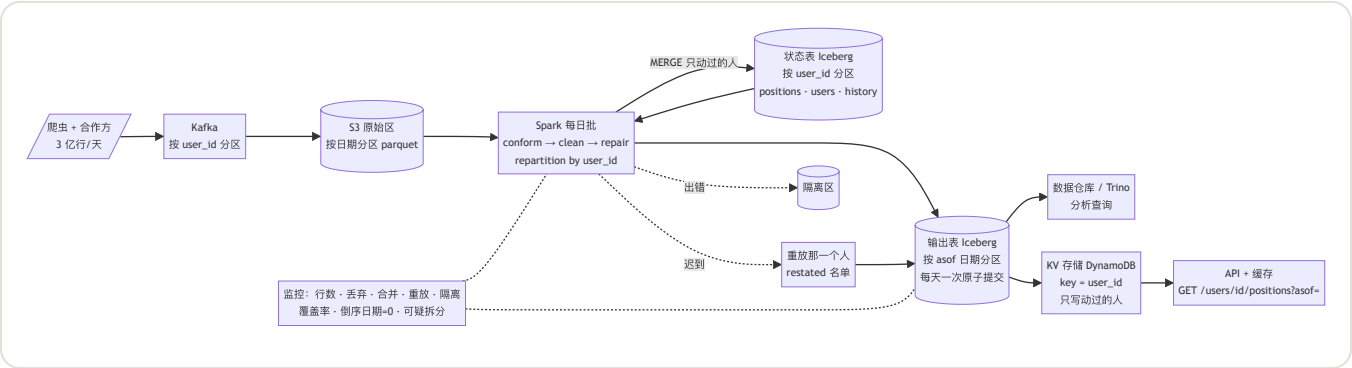
押题最终答案（三道，每道一张图 + 最优设计）

补充手册第 2 章有思路，这里给的是「最后要落到的那张图」。答不出来就画图、按框念。

题 A · 每天几亿条档案进来，维护去重后的「每人职业经历」（= 你的 take-home 上生产）

先问：每天多少条（设 3 亿）、新鲜度（天级够）、下游谁读（分析 + 点查）、要不要保留历史版本（要，as-of）。

估算：3 亿行/天 × 500 B = 150 GB/天；存量 1000 亿行 ≈ 50 TB；每天只有约 1% 的人有新数据；按 3.5 万行/核秒，每天增量 3 亿行 ≈ 2.4 核时，几台机器半小时。

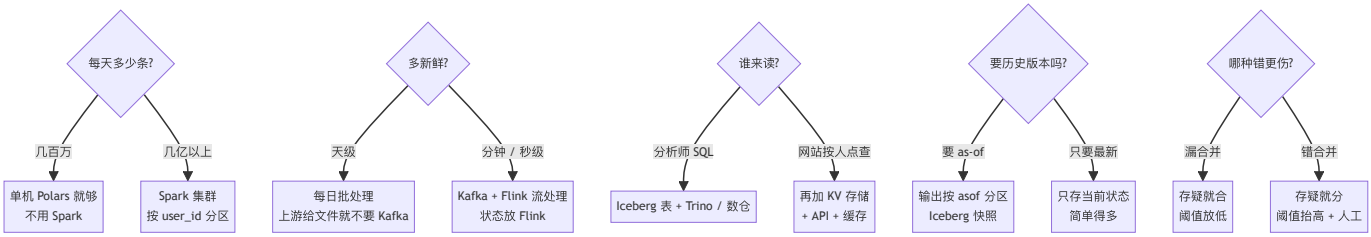


- **核心**：和作业一样的 coalesce_bucket，Spark mapInArrow 调同一个函数；按 user_id 分区所以状态 join 不用跨机器。
- **「输出表按 asof 日期分区」是什么意思**：asof_ts 是你输出里的那一列，「截至哪一天」的履历。按它分区 = 把输出表按日期分成一个个文件夹：`asof=2026-10-28/`、`asof=2026-10-29/`、`asof=2026-10-30/`，每天的结果放自己的文件夹里。比方：档案室按日期分抽屉，每天一个抽屉。好处：① 每天跑完只往今天的抽屉里放东西，不碰旧抽屉；② 查「某人 10 月 29 日的履历」只开那一个抽屉（分区裁剪，partition pruning），不用翻全部；③ 跑坏了只重做今天的抽屉。和「按 user_id 分区」的区别：状态表要按人找，所以按 user_id 分；输出表下游是按日期查，所以按日期分。Partitioning the output by asof date means one folder per day; each run only writes today's folder, a query for one day reads only that folder, and a bad day can be rewritten alone. State is partitioned by user because it's looked up by user; output is partitioned by date because it's queried by date.
- **状态**：Iceberg 按 user_id 分区、只 MERGE 动过的人、一次运行一次提交、旧快照留着回退；history 只追加、只给迟到的人读。
- **迟到**：事件时间 observed_ts；比状态旧的只重放那一个人，restated 告诉 KV 加载器整体替换。
- **隔离**：一个人出错回滚进隔离区，不阻塞当天运行。

- **服务**：分析走仓库；点查走 KV（key = user_id，value = 当前履历 + as-of 索引）+ 读缓存。
- **扩容**：人独立 → 加 executor 线性；分区数调到每分区几十万行；超级用户单独处理。
- **取舍**：天级批而不是流（新鲜度要求天级，流的乱序和状态管理不值）；规则而不是 LLM 逐对（可复现、可解释、1000 亿行成本）；merge when in doubt。
- **隐私（right to be forgotten）**：删除名单在输出时过滤。

Same core as the take-home, run under Spark partitioned by user_id, with state in an Iceberg table partitioned by user_id so each day merges only touched users and commits atomically. Late data replays one person; failures are quarantined. Output goes to a warehouse for analysis and a key-value store for point lookups, and every run emits the quality metrics I alert on.

先问的思维导图：问五个问题，每个答案决定一个组件。 左边是问题，箭头上是他可能的回答，右边是这个回答把设计引向哪里。



题 A：逐步思考和取舍（每一步：要解决什么 → 可选方案 → 选哪个 → 为什么 → 代价）

第 1 步 · 先问清楚要求（clarify requirements）

要解决什么：同一道题，每天 1 万条和每天 10 亿条、要秒级更新和要天级更新，设计完全不同。不问就画，十有八九画错方向。

可选方案：

- 问：每天进来多少条？（设 3 亿）
- 问：结果要多新鲜？秒级、小时级还是天级？（设天级，和 take-home 一样）
- 问：谁读结果？分析师跑 SQL，还是网站按人点查？（设两种都有）
- 问：要不要保留历史版本，能查「截至某天」的样子？（要，as-of）
- 问：错一点能不能接受？漏合并和错合并哪个更伤？

选哪个：问完把答案写在白板最上面，后面每个决定都回头指着它说「因为天级就够，所以……」。

为什么：面试官最看重你会不会先界定问题。每个答案都会直接决定后面一个组件的选择。

代价 / 取舍：花 3–5 分钟，看起来慢，但能避免后面整段推倒重来。

Before drawing anything, I'd confirm the daily volume, how fresh the output must be, who reads it, and whether we need history, because each answer changes a component.

第 2 步 · 估算（back-of-envelope）

要解决什么：知道数据有多大，才知道一台机器够不够、要几台、存储花多少钱。

可选方案：

- 只算数量级，不求精确

选哪个：3 亿行/天 × 500 字节 ≈ 150 GB/天；存量 1000 亿行 ≈ 50 TB；每天约 1% 的人有新数据；你的代码 3.5 万行/核秒 → 3 亿行 ≈ 8,600 核秒 ≈ 2.4 核时。

为什么：结论很关键：每天的增量很小，几台机器半小时跑完；真正大的是存量和偶尔的全量重算（800 核时）。所以设计重点是「增量只碰动过的人」，而不是「每天全量重跑」。

代价 / 取舍：估算有误差，所以说「数量级」，并说明假设（每行 500 字节、1% 的人有更新）。

Three hundred million rows a day at 500 bytes is about 150 gigabytes; at 35 thousand rows per core-second that's under three core-hours, so the daily job is small. The design should touch only users with new data rather than recompute everything.

第 3 步 · 数据怎么进来（ingest）

要解决什么：爬虫和合作方源源不断地送数据，要有个地方先接住，不能直接塞进处理程序（处理程序挂了数据就丢了）。

可选方案：

- 直接写数据库：简单，但数据库扛不住每天几亿行的写入，而且和处理逻辑绑死。
- 写进 Kafka（消息队列）：生产者和消费者解耦（decoupling），消费者挂了数据还在，可以重放（replay）。
- 直接落 S3 文件：最便宜，按天一个文件夹，但不方便实时消费。

选哪个：Kafka 按 user_id 分区接住 → 每小时把 Kafka 里的数据落成 S3 上按日期分区的 parquet（原始区，raw zone）。

为什么：Kafka 负责「接住、不丢、能重放」；S3 原始区负责「便宜地长期保存原样数据」，以后规则改了可以从原始数据重算。按 user_id 分区让同一个人的事件在同一个分区里保序。

代价 / 取舍：多维护一个 Kafka 集群。如果新鲜度只要天级、上游本来就每天给文件，可以不要 Kafka，直接收 S3 文件，更简单。面试时把这个取舍说出来。

I'd land events on Kafka partitioned by user_id so nothing is lost and one person's events stay in order, then persist them hourly as date-partitioned parquet in a raw zone on S3. If upstream already delivers daily files, I'd skip Kafka and read S3 directly.

第 4 步 · 选计算引擎：批还是流

要解决什么：每天要把新数据和已有状态合起来算一遍，用什么跑?

可选方案：

- Spark 每日批处理（batch)：攒一天的数据一次算完。
- Flink 流处理（streaming)：来一条算一条，秒级。
- 单机 Polars：你作业那样，够快但只有一台机器。

选哪个：Spark 每日批。

为什么：要求是天级新鲜度，流处理的好处（秒级）用不上，却要付出它的代价：一直开着、状态管理、乱序处理、运维（operations）重。Spark 成熟、按天跑正好、和 Iceberg 配合好，而且 mapInArrow 能直接调你写好的 coalesce_bucket，规则只有一份。

代价 / 取舍：新数据最晚要等一天才可见。如果以后要求变成分钟级，再把核心函数搬进 Flink（函数本身不用改，因为它就是「给一个人的状态和新快照，返回新状态」)。

Daily freshness makes batch the right call; streaming would add state management and operations for no benefit. Spark runs the same coalesce_bucket through mapInArrow, so there's one copy of the logic, and it could move to Flink later without changes.

第 5 步 · 核心处理怎么并行（partition by user_id）

要解决什么：1000 亿行一台机器跑不动，要分给很多机器。怎么分才不会算错?

可选方案：

- 随机分：均匀，但同一个人的行散在多台机器，没法做每人的状态机。
- 按 user_id 分：同一个人的数据一定在同一台机器。
- 按公司分：同一公司在一起，但一个人换过多家公司会被拆开。

选哪个：按 user_id 分区（repartition by user_id，这一步就是一次 shuffle）。

为什么：你的算法是「每人一个状态机」，人和人之间不共享任何东西，所以只要同一个人在一台机器上就能独立算，加机器就线性变快。

代价 / 取舍：一次 shuffle 要走网络，是最贵的一步；所以状态表也按 user_id 分区，让 join 时同一个人的状态和新数据本来就在一起，少搬数据。超级用户（几千条的人）可能造成数据倾斜，单独处理。

Partitioning by user_id puts all of one person's rows on one worker, and since nothing is shared between people, it scales linearly. The shuffle is the expensive step, so the state table is partitioned the same way.

第 6 步 · 状态存哪里（state store）

要解决什么：每个人「目前有哪些职位、下一个序号、上次处理到哪天」要存下来，明天接着用。

可选方案：

- 现在作业里的做法：每天把 parquet 整份重写到新文件夹。简单，但每天重写全部，哪怕只有 1% 的人变了。
- 放 KV 数据库（DynamoDB）：按人读写快，但批处理一次读几亿人很贵。
- 放 Iceberg 表，按 user_id 分区。

选哪个：Iceberg 表，按 user_id 分区，每天用 MERGE 只更新动过的人；history 单独一张只追加（append-only）的表。

为什么：① MERGE 只改动过的人，不用整份重写；② 按 user_id 分区，当天只读有新数据的人所在分区（分区裁剪，partition pruning）；③ 一次运行一次原子提交，跑一半挂了不会留下半截状态；④ 旧快照留着，跑坏了一键回退到昨天。

代价 / 取舍：多了一层表格式要维护（定期合并小文件 compaction、清理旧快照）。

State lives in an Iceberg table partitioned by user_id: each day merges only touched users, reads only their partitions, commits atomically, and keeps snapshots so a bad run can be rolled back.

第 7 步 · 迟到数据和出错怎么处理

要解决什么：有的快照晚几天才到；有的人数据奇怪会让程序报错。不能让一个人拖垮整批。

可选方案：

- 迟到：直接丢弃（简单但输出错）
- 迟到：隔离等人工（安全但积压）
- 迟到：只重放那一个人（你作业的做法）
- 出错：整批失败重跑 vs 只隔离出错的人

选哪个：迟到只重放那个人、写进 restated 名单；出错的人回滚进隔离区（quarantine），其余照常提交。

为什么：人和人独立，所以重放一个人、隔离一个人都不影响别人。重放结果和全量跑一致，有测试证明。

代价 / 取舍：要多存 history；restated 的人下游要整体替换他的历史行。隔离区要有人定期看。

Late snapshots replay just that user and are listed as restated; a user who fails is rolled back and quarantined while everyone else commits.

第 8 步 · 结果怎么给下游用（serving）

要解决什么：分析师要跑 SQL 统计；网站要「给我这个人截至某天的履历」毫秒级返回。一个存储满足不了两种读法。

可选方案：

- 只放数据仓库：SQL 方便，但点查要几秒。
- 只放 KV：点查快，但没法做统计。
- 两份都放。

选哪个：输出 Iceberg 表（按 asof 日期分区）给 Trino / 数仓跑分析；同时每天把动过的人批量写进 DynamoDB（key = user_id），前面加 API 和缓存给网站点查。

为什么：读法不同就用不同的存储，这叫读写分离 / 多存储（polyglot persistence）。每天只写动过的人，KV 的写入量很小。

代价 / 取舍：同一份数据两份拷贝，要保证一致：以 Iceberg 为准，KV 是它的派生，出问题从 Iceberg 重新灌。

Analysts query the Iceberg output through a warehouse; point lookups go to a key–value store loaded daily with only the touched users, behind an API and cache. The table is the source of truth and the store can be rebuilt from it.

第 9 步 · 监控（monitoring）

要解决什么：没有标准答案，上线后怎么知道结果还对、哪天抓取器坏了？

可选方案：

- 只看程序有没有报错（不够，结果错了程序不会报错）
- 每次运行输出质量指标并告警

选哪个：每次运行出指标：输入行数、丢弃行数（按规则）、修复格数、合并数、重放人数、隔离人数、覆盖率、倒序日期数、可疑拆分数、新 id 比例；和历史比，突变就告警；必须为 0 的（主键重复、倒序日期）直接让任务失败。再加定期抽样 + 判官。

为什么：数据错误不会抛异常，只能靠指标发现。抓取器改版通常最先表现为「新 id 比例突然涨了」。

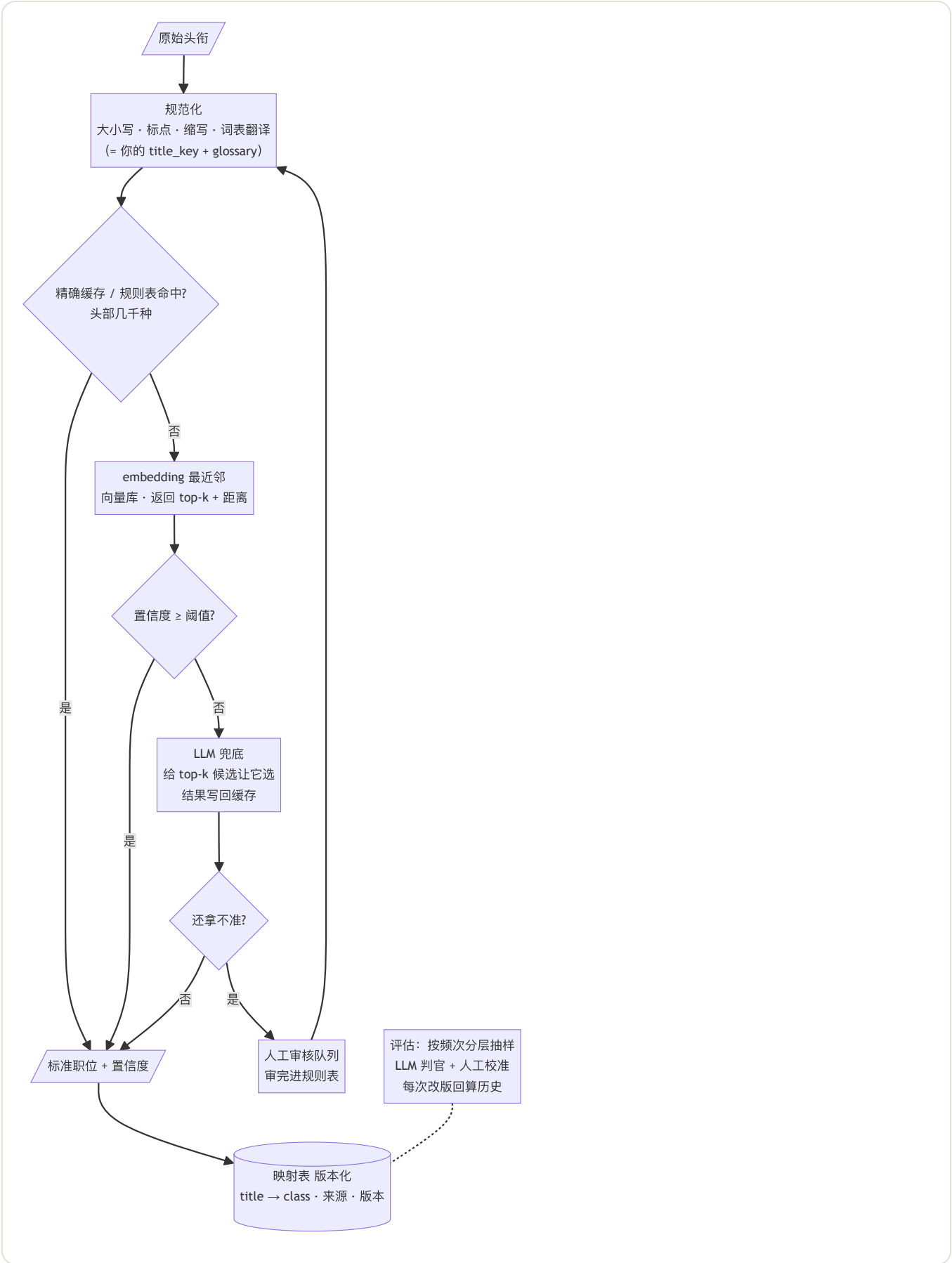
代价 / 取舍：告警阈值要调，太松漏报、太紧吵人。

Every run emits counts per rule, coverage, inverted dates, suspicious splits and the new-id rate; I alert on deviations from history and hard-fail on anything that must be zero, plus periodic sampled judging.

题 B · 职位名 / 技能映射服务（1000 万种原始职位名 → 标准分类）

先问：分类表多大（几千类）、延迟要求（线上毫秒 vs 离线批）、准确率目标、要不要多语言、分类表会不会改版。

估算：去重后约 1000 万种头衔，但头部几千种覆盖 80% 流量；线上 QPS 设 1 万；每个向量 384 维 × 4 B ≈ 1.5 KB，1000 万 × 1.5 KB = 15 GB，一台机器放得下。

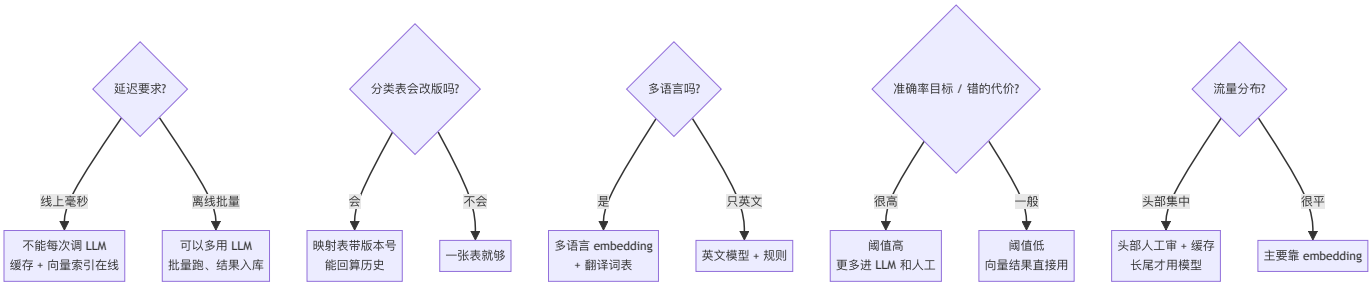


- **三层漏斗**：规则/缓存吃掉大头（毫秒、确定）；embedding 吃长尾（几毫秒）；LLM 只处理低置信（贵、慢、结果缓存后不再调）。

- **为什么不全用 LLM**：1000 万次调用成本，答案不稳定，线上延迟；LLM 的价值是兜底 + 当判官。
- **版本化**：分类表改版要能回算历史，不然时间序列断；映射表记来源（规则 / 向量 / LLM / 人工）。
- **评估**：按频次分层抽样（头部错一个影响大）；LLM 判官用人工样本校准；监控「进 LLM 的比例」「进人工的比例」，涨了说明有新词。
- **多语言**：多语言 embedding 模型；词表先翻译。
- **取舍**：准确 vs 延迟 vs 成本，用置信度阈值调；阈值高 → 更多进 LLM/人工。

A three-tier funnel: normalise, then rules and a cache for the head of the distribution, embedding nearest-neighbour for the long tail, and an LLM only for low-confidence cases, with results cached and the lowest sent to human review. The mapping table is versioned so history can be recomputed, and I evaluate on frequency-stratified samples with an LLM judge calibrated against human labels.

先问的思维导图：问五个问题，答案决定「花多少钱在准确率上」。 左边是问题，箭头上是他可能的回答，右边是这个回答把设计引向哪里。



题 B：逐步思考和取舍（每一步：要解决什么 → 可选方案 → 选哪个 → 为什么 → 代价）

第 1 步 · 先问清楚

要解决什么：职位名有一千万种写法，要映射到几千个标准类。线上实时用还是离线批量用、准确率要多少，决定了能不能用贵的方法。

可选方案：

- 问：标准分类表多大？会不会改版？
- 问：延迟要求？线上搜索框要毫秒级，离线报表可以慢
- 问：多语言吗？
- 问：准确率目标？错了的代价？

选哪个：设：几千个标准类、会改版；线上要毫秒级、离线也要；多语言；头部准确率要很高。

为什么：延迟和成本决定能不能每次都调 LLM（不能）。改版决定映射表要版本化。

代价 / 取舍：—

I'd confirm the taxonomy size and how often it changes, the latency target, languages, and the accuracy bar, since those decide whether we can afford an LLM per lookup.

第 2 步 · 估算

要解决什么：看数据分布，决定在哪里花钱。

可选方案：

- 统计每个头衔出现多少次

选哪个：一千万种头衔，但头部几千种覆盖约 80% 的流量（长尾分布，long tail）；线上 1 万 QPS；向量 384 维 × 4 字节 ≈ 1.5 KB，一千万个 ≈ 15 GB，一台机器内存放得下。

为什么：既然大头集中在少数头衔，就把它们做到又快又准；长尾才用更贵的方法。

代价 / 取舍：—

The distribution is long-tailed: a few thousand titles cover most traffic, so the head gets cheap exact handling and only the tail needs expensive methods.

第 3 步 · 规范化（normalisation）

要解决什么：「Sr. Software Engineer」「senior software engineer」「高级软件工程师」其实是一回事，先把写法统一，后面才好比。

可选方案：

- 什么都不做直接比（命中率低）
- 规范化：大小写、标点、缩写展开、词表翻译（你的 title_key + glossary）

选哪个：先规范化，生成一个 key。

为什么：规范化便宜、确定，一下子把很多写法合成同一个 key，后面缓存命中率大幅提高。

代价 / 取舍：规则要维护；翻译过度会把不相关的合在一起（你作业里 chief 被误翻那次）。

Normalise first, case, punctuation, abbreviations and a translation glossary, so many spellings collapse to one key before anything expensive runs.

第 4 步 · 头部：缓存 / 规则表

要解决什么：最常见的几千个头衔要毫秒级、百分百确定。

可选方案：

- 每次都现算
- 人工审过的规则表 + 缓存（Redis）

选哪个：规范化后的 key 先查规则表 / 缓存，命中直接返回。

为什么：占了大部分流量；人工审过一次就永远对；内存查找微秒级。

代价 / 取舍：规则表要有人维护；分类表改版时要批量更新。

The head is served from a reviewed mapping table in a cache: microseconds and exact.

第 5 步 · 长尾：embedding 最近邻

要解决什么：没见过的写法，规则对不上，需要「按意思找最像的标准类」。

可选方案：

- 编辑距离 / token 相似度（便宜，但不懂意思：Developer 和 Engineer 字面不像）
- embedding 最近邻（懂意思，几毫秒）
- 直接问 LLM（最懂，但贵、慢、不稳定）

选哪个：用多语言 embedding 模型把头衔变成向量，在标准类的向量里找最近的 top-k，并给一个距离当置信度。

为什么：embedding 懂语义、支持多语言、几毫秒、成本低，能吃掉大部分长尾。

代价 / 取舍：要部署模型和向量索引；距离阈值要用标注数据调。

For the long tail, a multilingual embedding finds the nearest standard titles in milliseconds, with distance as a confidence score.

第 6 步 · 低置信：LLM 兜底 + 人工

要解决什么：embedding 拿不准的（最近的两个类差不多近），需要更聪明的判断。

可选方案：

- 全交人工（准但慢、贵）
- 全交 LLM（贵、不稳定）
- 只把低置信的交 LLM，LLM 也拿不准的交人工

选哪个：低于阈值的交给 LLM：给它 top-k 候选让它选（不让它自由发挥）；结果写回缓存，同一个头衔不会问第二次。LLM 也拿不准的进人工队列，审完写进规则表。

为什么：把贵的资源只用在难的少数；LLM 结果缓存后成本是一次性的；人工审核的结果回流，系统越用越准。

代价 / 取舍：LLM 答案要可复现：固定模型版本、温度 0、记录版本号。

Only low-confidence cases go to an LLM, constrained to the top candidates and cached; the hardest go to human review, which feeds back into the rule table.

第 7 步 · 版本化和评估

要解决什么：分类表改版后，历史数据要能按新版本重算，否则时间序列断开；还要知道准确率有没有下降。

可选方案：

- 映射表只存最新（简单，但历史对不上）
- 映射表带版本号和来源（规则 / 向量 / LLM / 人工）

选哪个：映射表存 (头衔, 标准类, 来源, 分类表版本, 模型版本)；按频次分层抽样评估，LLM 判官用人工样本校准；监控「进 LLM 的比例」「进人工的比例」。

为什么：头部错一个影响大，所以分层抽样；比例突然上涨说明出现了新词或者模型退化。

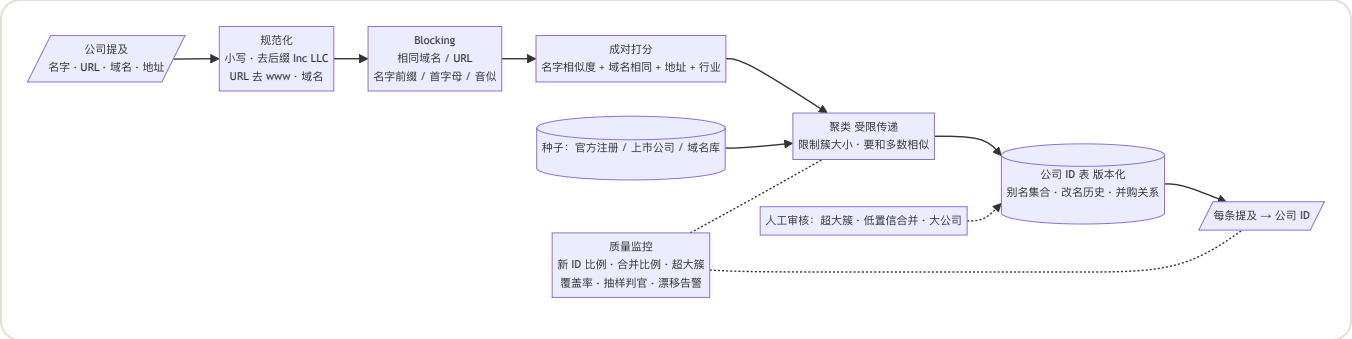
代价 / 取舍：多存几列；评估要持续投入人工。

The mapping is versioned with its source and taxonomy version so history can be recomputed, and I evaluate on frequency-stratified samples and watch the share sent to the LLM and to humans.

题 C · 公司实体消歧 + 数据质量监控

先问：公司实体来源（档案里的公司名、URL、官网、注册信息）、要合并到什么粒度（法人 vs 品牌 vs 子公司）、改名/并购怎么算、下游要什么（公司级统计）。

估算：几亿条公司提及，去重后几千万个名字，真实公司几百万；两两比不可能 → 必须 blocking。

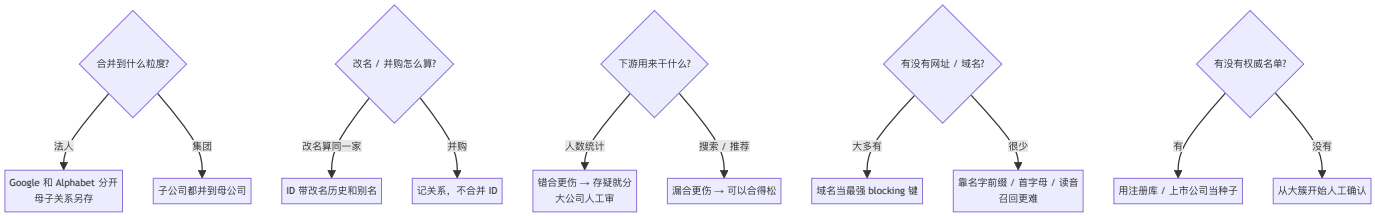


- **三步**：blocking（域名最强、名字前缀 / 首字母 / 音似兜底）→ 成对打分（名字 + 域名 + 地址 + 行业）→ 受限聚类（限制簇大小，防 A 像 B、B 像 C 的链式乱合）。
- **别名集合 + 改名历史**：一个公司记所有叫过的名字和域名；并购是关系不是合并（Jeppesen/Boeing 那种）。
- **种子数据**：官方注册库、上市公司、域名库当锚点，先把大公司钉死。
- **质量监控**：每天看新 ID 比例、合并比例、最大簇大小、覆盖率；分层抽样 + 判官；突变告警（抓取器变了会先体现在这里）。
- **人工**：只审三类：超大簇、低置信合并、头部大公司。
- **版本化**：ID 永不复用、可退役；历史可回算。
- **取舍**：合并错（两家公司算一家，统计错）比漏合并（一家算两家）更伤 → 这里和作业相反，存疑就分开，靠人工和种子来合。

Standard record linkage: normalise names and domains, block on domain and name prefix, score pairs on name similarity plus domain, address and industry, then cluster with a cap on cluster size to avoid chaining. Each company keeps an alias set and rename history; mergers are relationships, not merges. Official registries seed the big companies. For quality I track new-id and merge rates, max cluster size and coverage daily, sample with judges, and route oversized or low-confidence clusters to human

review. Here a wrong merge hurts more than a missed one, so unlike the take-home I split when in doubt.

先问的思维导图：问五个问题，最关键的是第 3 个：错合和漏合哪个更伤。 左边是问题，箭头上是他可能的回答，右边是这个回答把设计引向哪里。



题 C：逐步思考和取舍（每一步：要解决什么 → 可选方案 → 选哪个 → 为什么 → 代价）

第 1 步 · 先问清楚

要解决什么：「公司」的定义不同，答案完全不同：Google 和 Alphabet 算不算一家？YouTube 呢?

可选方案：

- 问：合并到什么粒度？法人、品牌、还是集团?
- 问：改名、并购怎么算?
- 问：下游用来干什么？公司人数统计最怕什么错?

选哪个：设：按法人合并；改名算同一家；并购记成关系（母子公司）不合并；下游做公司人数统计。

为什么：下游是统计人数，两家公司被错合成一家会让人数直接翻倍，比「一家被拆成两家」更伤。这决定了后面阈值要偏保守。

代价 / 取舍：—

I'd pin down the entity level, legal entity versus brand versus parent, and how renames and acquisitions count, because a wrong merge doubles a company's headcount.

第 2 步 · 估算 + 为什么要 blocking

要解决什么：几千万个不同的公司名，两两比较是多少次?

可选方案：

- 全部两两比
- 先分组再比

选哪个：几千万个名字两两比 ≈ 10^{14} 到 10^{15} 次，算不完。必须先用 blocking 把候选缩小到每组几十个。

为什么：blocking 是这类问题能不能做出来的关键。

代价 / 取舍：blocking 太严会漏掉本该合并的（召回下降），太松又算不完。所以用几种键各分一遍，取并集。

Comparing every pair is about ten to the fifteen comparisons, so blocking on cheap keys is what makes it feasible; I'd use several keys and union the candidates to protect recall.

第 3 步 · 规范化 + blocking 键

要解决什么：同一家公司有很多写法：Google LLC / google inc / google.com。

可选方案：

- 只比名字
- 名字 + 域名 + URL + 地址

选哪个：规范化：小写、去标点、去 Inc/LLC/Corp、URL 去 www 和 https、取域名。blocking 键：① 相同域名（最强）② 名字前两个词相同 ③ 首字母缩写相同（RL ↔ Revelio Labs）④ 读音相似。

为什么：域名几乎是公司的身份证，最可靠；名字类的键兜住没有网址的情况。

代价 / 取舍：多个键会产生更多候选，计算量上去一些。

Normalise names and URLs, then block on shared domain first, plus leading name tokens, initials and phonetic keys for records without a URL.

第 4 步 · 成对打分（pairwise scoring）

要解决什么：每个候选对，算「是同一家」的可能性。

可选方案：

- 手写规则打分（像你的 take-home）
- 训练一个分类器（逻辑回归 / 梯度提升树）

选哪个：先规则：名字相似度 + 同域名 + 地址 + 行业 + 员工共现（两个名字下面有同一批人）。有了标注后再训一个分类器输出概率。

为什么：规则可解释、马上能上线；分类器在有标注后能把信号组合得更好。

代价 / 取舍：分类器需要标注、要做版本管理和监控。

Score candidate pairs on name similarity, shared domain, address, industry and shared employees; start with rules and move to a trained classifier once labels exist.

第 5 步 · 聚类（clustering）

要解决什么：A 和 B 像、B 和 C 像，那 A、B、C 是不是一家?

可选方案：

- 直接传递合并（transitive closure）：简单，但会链式乱合，最后把一大堆不相关的公司合成一团
- 受限聚类：限制簇大小、要求新成员和簇里多数成员都像

选哪个：受限聚类，加上超大簇报警。

为什么：链式乱合是实体消歧最常见的灾难，下游人数会突然暴涨。

代价 / 取舍：会漏掉一些真该合并的长链，交给人工处理。

I'd cluster with constraints, a size cap and majority agreement, because plain transitive closure chains unrelated companies together.

第 6 步 · 种子数据和 ID 管理

要解决什么：怎么让大公司先对？公司 ID 怎么保持稳定？

可选方案：

- 从零开始全靠算法
- 用官方注册库、上市公司名单、域名库当锚点（seed）

选哪个：先用种子把大公司钉死；每个公司 ID 带别名集合（所有叫过的名字和域名）和改名历史；并购记成母子关系；ID 永不复用、可退役。

为什么：大公司影响最大，用权威数据锚定最划算；稳定 ID 让下游的时间序列不断。

代价 / 取舍：种子数据要买或者维护。

Seed the big companies from official registries, keep alias sets and rename history per company id, record acquisitions as relationships, and never reuse ids.

第 7 步 · 质量监控和人工

要解决什么：上线后怎么知道没合错?

可选方案：

- 只看程序有没有报错
- 每天看指标 + 抽样 + 人工审核关键部分

选哪个：每天看：新 ID 比例、合并比例、最大簇大小、每家公司人数的日变化（大公司门槛更低）；分层抽样 + 判官；人工只审三类：超大簇、低置信合并、头部大公司。

为什么：错合会表现为某家公司人数突然暴涨，最容易从指标上看出来。

代价 / 取舍：人工有成本，所以只审最关键的。

I'd track new-id and merge rates, max cluster size and daily headcount changes per company, sample with judges, and send oversized clusters, low-confidence merges and top companies to human review.

第 8 步 · 总取舍：存疑时合还是分

要解决什么：和你的 take-home 正好相反。

可选方案：

- 存疑就合（take-home 的选择）
- 存疑就分

选哪个：存疑就分开，靠种子和人工来合。

为什么：take-home 里错合只丢一个头衔，漏合是一个重复 id 永远带着；这里错合会让两家公司人数加在一起，统计全错，代价反过来了。

代价 / 取舍：会有一些重复的公司 ID，需要人工慢慢合。

Here I'd split when in doubt, the opposite of the take-home, because a wrong merge corrupts headcounts while a missed merge only leaves a duplicate to fix later.